

UNION, INTERSECT, MINUS – Relational operators

UNION, INTERSECT, MINUS are set operators that can be applied on records coming from 2 or many queries. The structure of the Selects must be the same!

UNION combines the results of two queries and eliminates duplicate selected rows.

Ex1: Calculate the discount for each customer, based on the number of products he bought, using the union set operator.

- If they bought between 100 and 1000 products, they should get a discount of 10% of the total value of their orders
- If they bought more than 1000 products and less than or equal to 2000, they should get a discount of 15% of the total value of their orders
- If they bought more than 2000 products, they should get a discount of 20% of the total value of their orders

```
SELECT cust_first_name, cust_last_name, SUM(quantity) no_prod,
```

```
0.1*SUM(quantity*unit_price) discount
```

```
from customers c, orders o, order_items oi
```

```
where c.customer_id=o.customer_id and o.order_id=oi.order_id
```

```
group by cust_first_name, cust_last_name
```

```
having SUM(quantity) between 100 and 1000
```

```
UNION
```

```
SELECT cust_first_name, cust_last_name, SUM(quantity) no_prod,
```

```
0.15*SUM(quantity*unit_price) discount
```

```
from customers c, orders o, order_items oi
```

```
where c.customer_id=o.customer_id and o.order_id=oi.order_id
```

```
group by cust_first_name, cust_last_name
```

```
having SUM(quantity)>1000 and SUM(quantity)<=2000
```

```
UNION
```

```
SELECT cust_first_name, cust_last_name, SUM(quantity) no_prod,
```

```
0.2*SUM(quantity*unit_price) discount
```

```
from customers c, orders o, order_items oi
```

```
where c.customer_id=o.customer_id and o.order_id=oi.order_id
```

```
group by cust_first_name, cust_last_name
```

```
having SUM(quantity) >2000;
```

Ex2: Display the products and the orders they appear on, but also, the products that were not ordered and the orders without products (if any, by mistake).

MINUS operator returns only unique rows returned by the first query but not by the second.

Ex3: Display the products that have a list price between 100 and 200, but are not orderable.

```
SELECT product_id, product_name, product_status, list_price from product_information
```

```
WHERE list_price BETWEEN 100 AND 200
```

MINUS

```
SELECT product_id, product_name, product_status, list_price from product_information
```

```
WHERE product_status='orderable';
```

Ex4: Display the departments in which no one works.

INTERSECT operator returns only those rows returned by both queries.

Ex5: Display the products (product_name), their total ordered value, the number of orders they appear in, only for the products that have the total ordered value larger than 10000 and were ordered less than 10 times.

```
SELECT product_name, sum(quantity*unit_price) value, COUNT(order_id) no_orders
```

```
from product_information p, order_items o
```

```
where p.product_id=o.product_id
```

```
group by product_name
```

```
HAVING SUM(quantity*unit_price)>10000
```

```
intersect
```

```
SELECT product_name, sum(quantity*unit_price) value, COUNT(order_id) no_orders
```

```
from product_information p, order_items o
```

```
where p.product_id=o.product_id
```

```
group by product_name
```

```
HAVING COUNT(order_id)<10;
```